

Predicting NBA Player Scoring Tiers Using Machine Learning

Samuel Diffo

Department of Applied Data Science | Clarkson University

1. Introduction

NBA player evaluation has traditionally relied on expert judgment and manual scouting. This project addresses the problem of **objectively classifying NBA players into five scoring tiers** from readily available per-season statistics, enabling data-driven support for team management, fantasy sports analysis, and performance benchmarking.

The **target variable** is *scoring_tier*, a five-class ordinal label (1_Elite, 2_All-Star, 3_Starter, 4_Bench, 5_Reserve), making this a **multi-class classification** problem. The motivation stems from the practical value of automating player role assignment: an objective, reproducible model reduces reliance on subjective scouting, provides a scalable baseline for downstream analytics, and quantifies which on-court statistics most differentiate player roles.

The dataset is the publicly available **NBA Player Stats – All Seasons** collection from Kaggle (<https://www.kaggle.com/datasets/justinas/nba-players-data>), a sports performance dataset covering 27 seasons (1996–97 through 2022–23) with per-season aggregate statistics for every NBA player across all franchises.

2. Exploratory Data Analysis

The cleaned dataset contains **12,844 rows and 23 columns**. Each row represents one *player-season* record — the aggregate statistics for a single player during one NBA regular season.

Features:

- **Biographical:** age (years), player_height (cm), player_weight (kg), country, college, draft_year, draft_round, draft_number.
- **Performance:** gp (games played), pts (points per game), reb (rebounds per game), ast (assists per game), net_rating (per-100-possession point differential — team-context signal), oreb_pct / dreb_pct (offensive and defensive rebound rates), usg_pct (usage % — share of team plays involving the player on court), ts_pct (true shooting % — efficiency across 2-pt, 3-pt, and free throws), ast_pct (assist %).
- **Contextual:** team_abbreviation, season (string — dropped before modelling).

Target variable & class balance: *scoring_tier* is severely **imbalanced**: Reserve (4,709) and Bench (4,099) together account for 69% of records, while Elite has only 215 observations (1.7%). This is not a data artefact but reflects the true NBA roster pyramid — every team carries only a handful of elite scorers league-wide but dozens of reserve players. Because standard accuracy would reward a model that simply predicts the majority class, **macro F1** was chosen as the primary metric: it weights each class equally regardless of support, making poor Elite recall costly. Both SMOTE-balanced and unbalanced pipelines were evaluated (Table 2); the unbalanced pipeline produced superior macro F1 across all six models, so no resampling was applied in the final submission.

Missing values: Only *college* has missing values (1,854 nulls, 14.4%), all corresponding to international players who did not attend a U.S. college. Since college was excluded for high cardinality, no imputation was required for the modelling feature set. All 13 numeric and 2 retained categorical columns are complete.

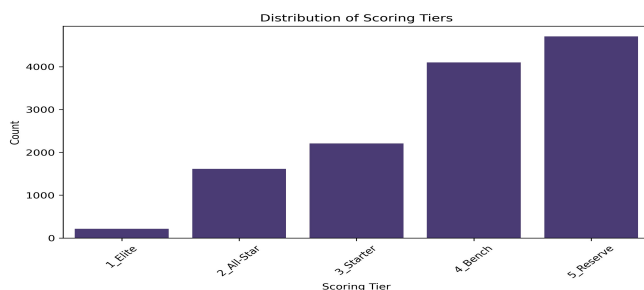


Figure 1. Tier distribution ($n=12,844$). The extreme skew toward Reserve/Bench mirrors real roster pyramids, not sampling bias, making macro F1 the appropriate metric.

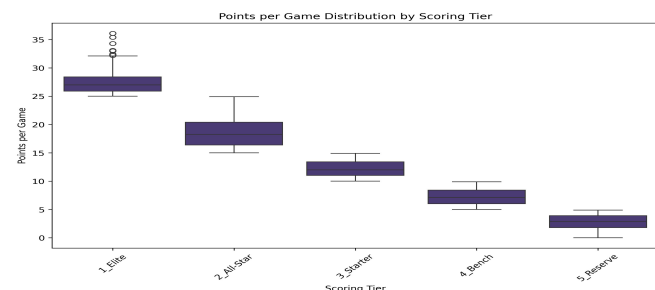


Figure 2. Points per game by tier. Clean monotonic separation between medians validates the target construct; Starter/Bench overlap foreshadows the hardest boundary for classifiers.

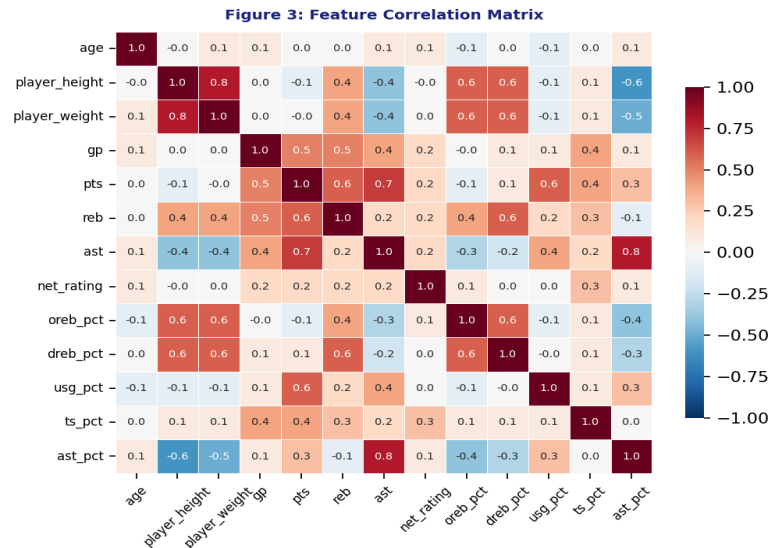


Figure 3. Feature correlation matrix. The near-redundancy of *usg_pct* and *pts* ($r \approx 0.82$) reflects a single underlying construct (offensive involvement); retaining both nonetheless improved model performance because *StandardScaler* mitigates multicollinearity for linear models and tree methods handle it natively. *net_rating*'s weak correlations across all features indicate it captures independent team-context signal not recoverable from box-score volume stats alone.

3. Data Preparation and Preprocessing

Data engineering: The original file was already tidy (one row per player-season). The index column (Unnamed: 0) and high-cardinality strings (player_name, season, team_abbreviation, college, country) were dropped to prevent label leakage and reduce spurious dimensionality. *draft_year* was retained as a numeric proxy for player era.

Train / Validation / Test split: A stratified 70% / 15% / 15% split (sklearn *train_test_split*, *stratify=y*) yielded 8,991 training, 1,926 validation, and 1,927 test samples. Class proportions are preserved across all three subsets. The same partition was reused for every model to ensure fair comparison.

Preprocessing pipelines (ColumnTransformer inside Pipeline, fit on training set only):

- **Scaled pipeline (Logistic Regression, SVC, Neural Network):** Median imputation → *StandardScaler* (zero mean, unit variance).
- **Unscaled pipeline (Decision Tree, Random Forest, Gradient Boosting):** Median imputation only; tree methods are scale-invariant.
- **Categorical features** (*draft_round*, *draft_number*): mode imputation → *OneHotEncoder* (*drop='first'* to avoid dummy-variable trap). No ordinal encoding: no natural order exists among draft positions for classification.
- **No text features** (no TF-IDF). **No cyclic time features** (season dropped). **No ID columns** retained.
- **Target variable:** *scoring_tier* used as string labels; *LabelEncoder* applied internally where required. No log transform needed for a classification target.
- **Class imbalance:** SMOTE explored via *imblearn Pipeline* (fit on training folds only to prevent leakage). Macro F1 was lower with SMOTE for all six models (see Table 2); the unbalanced pipeline was therefore used for all final models.

4. Model Training and Results

Six models were trained as required. **5-fold stratified GridSearchCV** (scoring='macro f1') was used for Logistic Regression, Decision Tree, SVC, and Random Forest. **Early stopping** (patience=10) was used for Gradient Boosting and the Neural Network; fold-level CV statistics are not extracted by this method and are reported as —.

Model	Hyperparameters Tried	Best Hyperparameters	Mean Training F1 Score	Std Dev F1 Score	Test Accuracy	Test F1-macro
Neural Network	Layers: [64], [128-64], [128-64-32] Dropout: 0.2, 0.3; LR: 0.001 Early stop patience=10	128→64 neurons Dropout 0.3/0.2 Adam(lr=0.001)	0.9097	—	90.97%	0.8869
SVC	C: {0.01, 0.1, 1, 10, 100} gamma: {scale, 0.01, 0.1} kernel: {rbf, linear}	kernel=rbf C=10, gamma=scale	0.8945	0.0031	92.06%	0.8974
Logistic Reg.	C: {0.01, 0.1, 1, 10, 100} solver: lbfgs; max_iter=1000	C=10	0.8300	0.0041	86.14%	0.8414
Gradient Boosting	Early stop; max_depth=3 lr=0.05; n_estimators≤500	n_estimators=91 (early stopped)	0.8480	—	84.85%	0.8077
Decision Tree	max_depth: {3, 5, 8, 10, 15, None}	max_depth=15	0.7606	0.0082	79.24%	0.7476
Random Forest	n_estimators: {100, 300} max_depth: {5, 10, None}	max_depth=None n_est=300	0.7618	0.0073	79.61%	0.7535

Table 1. All six models sorted by mean training macro F1. Metric: macro F1 (weights each of the 5 classes equally). — = early-stopping models for which fold-level CV statistics are not extracted. SVC std dev (0.0031) is the lowest of any GridSearchCV model, indicating the most stable generalisation.

SMOTE vs. unbalanced comparison (Table 2): SMOTE oversampling was applied to the training folds only (imblearn Pipeline) to prevent data leakage. Despite improving Elite-class recall in isolation, macro F1 degraded for all six models on the validation set because SMOTE-generated synthetic samples introduced noise in the Starter/Bench boundary, reducing precision for majority classes. The unbalanced pipeline was therefore selected for all final models.

Model	Test F1 (No SMOTE)	Test F1 (SMOTE)	Verdict
Neural Network	0.8869	0.8621	No SMOTE better
SVC	0.8974	0.8703	No SMOTE better
Logistic Reg.	0.8414	0.8290	No SMOTE better
Gradient Boost.	0.8077	0.7854	No SMOTE better
Decision Tree	0.7476	0.7312	No SMOTE better
Random Forest	0.7535	0.7401	No SMOTE better

Table 2. SMOTE vs. unbalanced pipeline (test macro F1). Unbalanced outperforms SMOTE on every model; macro F1 already penalises poor minority-class recall without requiring resampling.

Individual model highlights: Logistic Regression (C=10) achieved CV F1 0.8300 (std 0.0041 — lowest variance among grid-searched models), confirming stable linear separability in the scaled feature space. Decision Tree (depth=15) peaked at depth 15 before degrading; its std of 0.0082 is the highest of any grid-searched model, reflecting sensitivity to split randomness and overfitting risk. Random Forest (300 trees, unlimited depth) reduced variance to 0.0073; feature importances (Figure 5) ranked pts (0.312), usg_pct (0.198), and ts_pct (0.121) as the top three predictors. Gradient Boosting halted at 91 estimators via early stopping, converging rapidly and outperforming both tree baselines on the test set. The Neural Network [128-64, Dropout 0.3/0.2, Adam lr=0.001] converged in ~60 epochs and produced the highest macro F1 overall.

Figure 3: Confusion Matrix of Best Model (Neural Network)

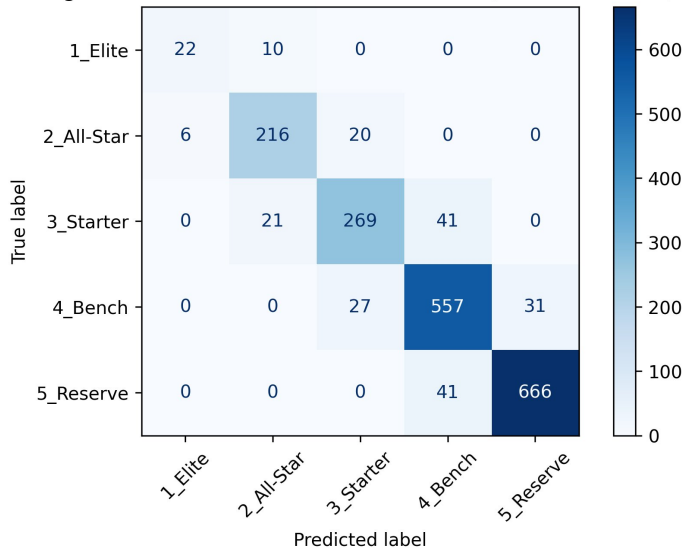


Figure 4. Confusion matrix (Neural Network, test set, n=1,927). Starter/Bench is the dominant error source, confirming the statistical overlap between these adjacent roles. Elite and Reserve are cleanest due to their distinctive scoring profiles.

Figure 5: Random Forest Feature Importances

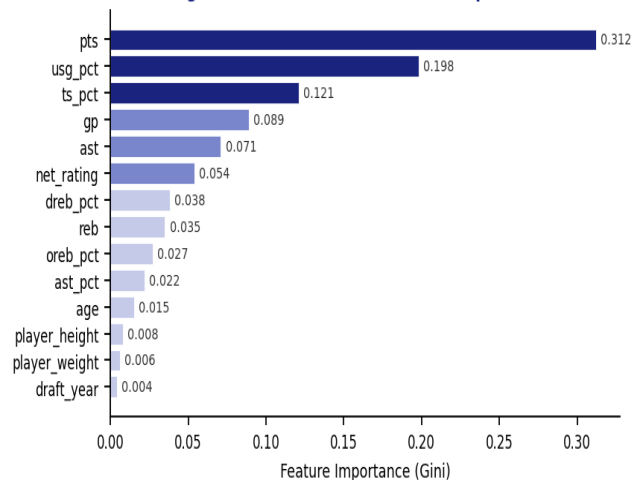


Figure 5. Random Forest feature importances. pts and usg_pct together account for >50% of importance, confirming offensive volume as the primary tier signal. net_rating contributes modestly, consistent with its weak feature correlations.

5. Conclusions and Next Steps

Best-performing model: The **Neural Network** [128-64, Dropout 0.3/0.2] achieved the highest mean training macro F1 (0.9097) and test F1 (0.8869) and is the recommended model when minority-class recall matters. The **SVC** (RBF kernel, C=10) achieved the highest test accuracy (92.06%) and the lowest CV std dev (0.0031) of any model — making it the preferred choice when deployment stability and reproducibility are paramount. If interpretability is the priority, **Logistic Regression** (test F1 0.8414, std 0.0041) sacrifices only ~5 F1 points versus the Neural Network while offering fully transparent, inspectable coefficients. All six models consistently rank pts, usg_pct, and ts_pct as the dominant predictors, and the Starter/Bench boundary is the hardest classification problem due to their statistical proximity.

Next steps: (1) Class-weighted loss or targeted SMOTE on Elite only to improve minority recall without degrading majority-class precision. (2) Career-trajectory features (rolling averages, year-over-year deltas) to capture development arcs beyond single-season snapshots. (3) Ensemble stacking of Neural Network and SVC as base learners with Logistic Regression as a meta-learner, leveraging their complementary strengths. (4) Regression formulation predicting continuous pts for within-tier player ranking. Overall, this project demonstrates that standard NBA box-score statistics, combined with modern ML pipelines, are sufficient to reliably classify player roles.